[ENGENHARIA REVERSA]

Aplicada a Sites e Aplicações Web

Análise de Frontend · Desofuscação · APIs · Decompilação · Segurança

---

■ Use apenas em ambientes com autorização explícita ou em seus próprios projetos.

#	Seção	Conteúdo
01	O que é Engenharia Reversa Web	Conceito, objetivos, casos de uso legítimos
02	Ferramentas do Browser	DevTools, Network tab, Sources, Performance
03	Análise de JavaScript	Unminify, desofuscação, source maps, bundlers
04	Análise de APIs	REST, GraphQL, WebSocket, interceptação
05	Autenticação & Tokens	JWT, OAuth, cookies, session — como analisar
06	WebAssembly (WASM)	Decompilação, análise de lógica em WASM
07	Aplicações React/Vue/Angular	SPA reverse engineering, rotas, estado
08	Source Maps	Recuperar código-fonte original de produção
09	Engenharia Reversa de APIs	Reconstruir documentação de APIs privadas
10	Checklist Prático	Passo a passo para auditar qualquer site

01 // O QUE É ENGENHARIA REVERSA WEB

Engenharia reversa web é o processo de analisar e compreender o funcionamento interno de uma aplicação web sem acesso ao código-fonte original. Diferente de aplicações desktop (onde você analisa binários), em aplicações web o 'binário' é o código JavaScript, HTML, CSS e as chamadas de rede — todos acessíveis pelo browser.

Casos de Uso Legítimos

Caso de Uso	Descrição
Pentest / Bug Bounty	Encontrar vulnerabilidades em aplicações autorizadas
Auditoria de segurança	Verificar se a lógica de negócio está exposta no frontend
Análise de malware web	Entender scripts maliciosos injetados em sites
Compatibilidade / integração	Entender APIs sem documentação para integrar sistemas
Pesquisa acadêmica	Estudar técnicas de ofuscação e proteção de código
Seus próprios projetos	Auditar seu próprio site antes de lançar
CTF / Wargames	Desafios de segurança como HackTheBox, TryHackMe

O que você encontra em uma aplicação web

- HTML/CSS — estrutura e estilos (sempre legíveis)
- JavaScript — lógica de frontend (minificado/ofuscado, mas acessível)
- Requisições de rede — toda comunicação com o backend (APIs, dados)
- Cookies e Storage — sessões, preferências, tokens de autenticação
- Source Maps — em muitos casos, o código-fonte ORIGINAL está disponível
- WebAssembly — módulos compilados para performance (mais difíceis de analisar)
- Metadados — comentários HTML, headers HTTP, erros reveladores

02 // FERRAMENTAS DO BROWSER (DevTools)

O Chrome/Firefox DevTools é sua principal arma. Acesse com F12 ou Ctrl+Shift+I. Cada aba revela uma camada diferente da aplicação.

Aba DevTools	O que analisar
Elements	HTML em tempo real, atributos ocultos, data-* attributes com info sensível
Console	Erros JS, logs de debug, executar código arbitrário no contexto da página
Sources	Todo o JS/CSS carregado, breakpoints, source maps, snippets
Network	Todas as requisições HTTP(S): APIs, payloads, headers, tokens
Application	Cookies, localStorage, sessionStorage, IndexedDB, Service Workers
Performance	Profiling de execução, identificar funções pesadas
Memory	Heap snapshots, objetos em memória (pode conter dados sensíveis)
Security	Certificados, HTTPS, mixed content

Console — Comandos Essenciais

```
// Ver todos os cookies document.cookie // Ver localStorage completo
JSON.stringify(localStorage) // Interceptar todas as requisições fetch (monkey patch) const
origFetch = window.fetch; window.fetch = function(...args) { console.log('[FETCH]', args);
return origFetch.apply(this, args); }; // Interceptar XMLHttpRequest const origOpen =
XMLHttpRequest.prototype.open; XMLHttpRequest.prototype.open = function(method, url) {
console.log('[XHR]', method, url); return origOpen.apply(this, arguments); }; // Ver variáveis
globais do app (React, Angular, Vue expõem estado) window.__REDUX_STORE__
window.__REACT_DEVTOOLS_GLOBAL_HOOK__ window.angular
```

Network Tab — O que procurar

- Filtre por XHR/Fetch para ver apenas chamadas de API
- Clique em qualquer requisição → aba 'Headers' para ver tokens de auth
- Aba 'Payload' revela o corpo da requisição (dados enviados)
- Aba 'Response' mostra o retorno do servidor (dados recebidos)
- Clique direito → 'Copy as cURL' para reproduzir a requisição no terminal
- Preserve log: marcar para não perder requisições durante redirecionamentos
- Disable cache: forçar download de todos os assets (evitar cache stale)

03 // ANÁLISE DE JAVASCRIPT

JavaScript moderno passa por bundlers (Webpack, Vite, Rollup) e minificadores que removem espaços, encurtam variáveis e tornam o código ilegível. Engenharia reversa de JS é o processo de recuperar a lógica original.

Unminify — Tornar o Código Legível

No DevTools:

```
1. Sources → abra qualquer arquivo .js minificado 2. Clique no ícone { } (Pretty Print) no canto inferior esquerdo 3. O código é formatado com indentação legível 4. Use Ctrl+F para buscar por: token, password, api, key, secret, auth
```

Ferramentas online e CLI:

```
# Online https://beautifier.io -- unminify JS/CSS/HTML https://deobfuscate.io -- desofuscação automática https://prettier.io/playground # CLI npm install -g js-beautify js-beautify app.min.js -o app_readable.js # prettier npx prettier --parser babel app.min.js > app_clean.js
```

Desofuscação de JavaScript

Muitos sites usam ofuscação intencional (proteção de IP). Técnicas comuns:

Técnica de Ofuscação	Como reverter
Variáveis com nomes aleatórios	Renomear pelo contexto ao entender a lógica
String splitting (a+b+c)	Executar no console: eval() ou copiar e rodar
eval() aninhado	Substituir eval por console.log para ver o código
Array de strings rotativas	Executar o decoder no console para ver strings reais
Hex/Unicode encoding	\x41\x42 = AB; \u0041 = A — converter direto
Obfuscator.io style	deobfuscate.io ou análise manual por execução
Dead code injection	Ignorar branches que nunca executam (always false)

Técnica: Executar para Desofuscar

```
// Se o código tem uma funcao decodificadora, chame ela: // Ofuscado: var _0x1a2b = ['cGFzc3dvcmQ=', 'dXNlcm5hbWU=']; function _0xdec(i){ return atob(_0x1a2b[i]); } // No console, descubra o que cada indice retorna: _0xdec(0) // → 'password' _0xdec(1) // → 'username' // Substituicao manual + renomear funcoes revela a logica real
```

Busca por Segredos no JS

```
# Baixar todos os JS do site wget -r -A '*.js' --no-parent https://alvo.com/ 2>/dev/null # Buscar por chaves e credenciais grep -rEi '(api[_-]?key|apikey|secret|password|passwd|token|aws_access|private_key|auth|bearer|client_secret)' *.js # Buscar endpoints de API embutidos grep -rEo '"(/api/|/v[0-9]/|/graphql|/rest/)[^"]+"' *.js # Ferramenta: truffleHog trufflehog filesystem ./js_files/ # Ferramenta: secretfinder python3 SecretFinder.py -i https://alvo.com/app.js -o results.html
```

04 // ANÁLISE E INTERCEPTAÇÃO DE APIs

A maioria dos sites modernos é uma SPA (Single Page Application) que consome uma API REST ou GraphQL. Entender essa API é entender o backend — mesmo sem acesso ao código do servidor.

Interceptação com Burp Suite

```
1. Configure o proxy: Burp → Proxy → Options → 127.0.0.1:8080 2. Instale o certificado CA do Burp no browser (para HTTPS) 3. Navegue pelo site — todas as requisicoes aparecem em HTTP History 4. Use o filtro para encontrar chamadas de API (/api/, /graphql, /v1/) 5. Clique direito → Send to Repeater para modificar e reenviar 6. Send to Intruder para fuzzing automatico de parametros 7. Extensions: Logger++, Param Miner, JS Miner, GraphQL Raider
```

mitmproxy — Proxy via Terminal

```
# Iniciar mitmproxy mitmproxy --listen-port 8080 # Modo web (interface no browser) mitmweb --listen-port 8080 # Salvar todo o trafego mitmproxy --listen-port 8080 -w trafego.dump # Script Python para filtrar/modificar requisicoes em tempo real mitmdump --listen-port 8080 -s filtro.py # filtro.py — logar apenas chamadas de API def request(flow): if '/api/' in flow.request.url: print(flow.request.method, flow.request.url) print(flow.request.text)
```

GraphQL — Introspecção

GraphQL permite consultar o schema completo da API — se a introspeccao estiver habilitada:

```
# Introspeccao completa (revela TODA a API) curl -X POST https://alvo.com/graphql \ -H "Content-Type: application/json" \ -d '{"query": "{ __schema { types { name fields { name type { name } } } } }"}' # Listar queries e mutations disponiveis curl -X POST https://alvo.com/graphql \ -H "Content-Type: application/json" \ -d '{"query": "{ __schema { queryType { fields { name description } } } }"}' # Ferramenta: GraphQL Voyager (visualizacao do schema) # Ferramenta: InQL (extensao Burp para GraphQL) # Ferramenta: graphw00f (fingerprint de engine GraphQL)
```

WebSocket — Análise

```
# DevTools: Network → WS (filtro WebSocket) # Clique na conexao → Messages para ver trafego em tempo real # wscat — cliente WebSocket CLI npm install -g wscat wscat -c wss://alvo.com/ws # Interceptar no Burp: suporte nativo a WS # HTTP History → filtrar por tipo WebSocket # Messages aparecem em tempo real na aba WebSockets
```

05 // AUTENTICACAO E TOKENS

JWT — Decodificacao e Analise

JWT (JSON Web Token) tem 3 partes separadas por ponto: HEADER.PAYLOAD.SIGNATURE Cada parte e codificada em Base64URL (nao criptografada!)

```
# Decodificar manualmente (Linux)
TOKEN='eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VyIjoiYWRtaW4ifQ.xxx' echo $TOKEN | cut -d.
-f1 | base64 -d 2>/dev/null # Header echo $TOKEN | cut -d. -f2 | base64 -d 2>/dev/null # Payload
# jwt.io — decodificar online (nao envie tokens reais!) # jwt_tool — analise completa python3
jwt_tool.py $TOKEN python3 jwt_tool.py $TOKEN -T # Tamper interativo python3 jwt_tool.py $TOKEN
-C -d wordlist.txt # Crack secret # O que procurar no payload: # - role, admin, isAdmin (tentar
mudar para true) # - exp (expiracao — token vencido ainda aceito?) # - sub/user_id (tentar IDOR)
# - alg (tentar mudar para 'none')
```

OAuth 2.0 — Fluxo e Analise

Intercepte o fluxo OAuth completo no Network tab:

```
1. Inicio do fluxo: GET /oauth/authorize?client_id=XXX&redirect;_uri=https://alvo.com/callback
2. Pontos de analise: - redirect_uri pode ser manipulado? - state parameter presente? (protecao
CSRF) - scope e minimo necessario ou excessivo? - code pode ser reutilizado? - access_token
expira rapidamente? 3. Teste de redirect_uri manipulation: Substitua por:
?redirect_uri=https://evil.com Se o token for para evil.com = vulnerabilidade critica
```

Cookies — Analise de Flags e Conteudo

Flag	Se ausente → risco
HttpOnly	JavaScript pode ler o cookie → XSS rouba sessao
Secure	Cookie trafega em HTTP claro → MITM
SameSite	Cookie enviado em requests cross-site → CSRF
Domain	Escopo muito amplo → subdomainios maliciosos acessam
Expires	Session cookie sem expiracao → sessao permanente

06 // WEBASSEMBLY (WASM)

WebAssembly e um formato binario de alta performance que roda no browser. Sites usam WASM para proteger logica critica (algoritmos de licenca, DRM, cripto). E mais dificil de analisar que JS, mas nao impossivel.

Identificar e Extrair WASM

```
# Network tab: filtrar por .wasm # Geralmente: app.wasm, game.wasm, crypto.wasm # Baixar o arquivo .wasm curl https://alvo.com/static/app.wasm -o app.wasm # Verificar magic bytes xxd app.wasm | head -1 # Deve comecar com: 00 61 73 6d (= .asm em ASCII)
```

Decompilacao de WASM

```
# wat (WebAssembly Text Format) — formato legivel wasm2wat app.wasm -o app.wat cat app.wat # ver o codigo em formato textual # wasm-decompile (wabt toolkit) wasm-decompile app.wasm -o app_decompiled.dcmp # Ghidra (com plugin WASM) # Importar .wasm → analise grafica de funcoes # Renomear funcoes pelo contexto # wasmtime — executar e debugar WASM wasmtime app.wasm # Ferramenta online: webassembly.studio # Ferramenta: Binary Ninja com plugin WASM
```

O que procurar em WASM

- Strings embutidas: chaves de API, URLs de backend, mensagens de erro
- Algoritmos de validacao de licenca (serial numbers, activation keys)
- Logica de criptografia proprietaria (pode ter fraquezas)
- Chamadas de funcao exportadas — interface entre JS e WASM
- Importacoes do environment — o que o WASM acessa no browser

07 // ENGENHARIA REVERSA DE SPAs

Single Page Applications modernas (React, Vue, Angular) compilam todo o codigo em bundles JS. Mesmo sem source maps, e possivel extrair muita informacao — rotas, componentes, chamadas de API, estado global.

React — Inspecao de Componentes

```
# React DevTools (extensao do browser) # Aba Components: arvore de componentes + props + state # Aba Profiler: performance de renderizacao # No console: acessar estado de componentes # Clique em um elemento → $0 referencia o DOM node const fiber = $0._reactFiber || $0.__reactFiber; fiber.return.memoizedProps // props do componente pai fiber.memoizedState // estado atual # Redux DevTools (se o app usa Redux) # Aba Redux: ver todas as actions e o state tree completo window.__REDUX_DEVTOOLS_EXTENSION__ window.__REDUX_STORE__.getState() // estado global completo
```

Extrair Rotas da SPA

```
# Buscar no JS compilado por padroes de rotas # React Router grep -rEo 'path: ?"[^"]+"' *.js | sort -u grep -rEo "path: ?'[^']+" *.js | sort -u # Buscar components e pages grep -rEo '"/(admin|dashboard|settings|api|internal)[^"]*"' *.js # Ferramenta: webpack-exploder # Revela a estrutura de modulos do bundle Webpack # Buscar por comentarios de debug deixados grep -rEi '(TODO|FIXME|HACK|XXX|debug|console\.log)' *.js
```

Angular — Analise Especifica

```
# Angular expoe o NgProbe na janela ng.probe($0) // inspecionar componente ng.getComponent($0) // obter instancia do componente ng.getComponent($0).someVar // acessar variaveis internas # Listar todos os services injetados ng.probe($0).injector.view.root.ngModule._providers # Angular DevTools (extensao Chrome) # Ver arvore de componentes + performance de change detection
```

Webpack Bundle Analysis

```
# webpack-bundle-analyzer (se tiver acesso ao projeto) npm install -g webpack-bundle-analyzer webpack-bundle-analyzer stats.json # Sem acesso: analise manual do bundle # Buscar por: __webpack_require__, __webpack_exports__ # Modulos sao separados por /***/ e numerados # Ferramenta: relative-ci/bundle-stats # Ferramenta: source-map-explorer npx source-map-explorer app.js app.js.map
```

08 // SOURCE MAPS — CODIGO ORIGINAL EM PRODUCAO

Source maps mapeiam o codigo minificado de volta ao codigo-fonte original. Muitos times deixam source maps acessiveis em producao por engano — isso expoe o codigo-fonte COMPLETO, incluindo comentarios, logica de negocio e segredos.

Detectar Source Maps Expostos

```
# Checar comentario no final do JS minificado curl https://alvo.com/static/app.js | tail -5 # Procurar por: //# sourceMappingURL=app.js.map # Tentar acessar o .map diretamente curl -I https://alvo.com/static/app.js.map curl -I https://alvo.com/static/main.chunk.js.map # Nuclei template para source maps expostos nuclei -u https://alvo.com -t exposures/files/javascript-source-map.yaml
```

Extrair Codigo Original do Source Map

```
# Baixar o arquivo .map curl https://alvo.com/static/app.js.map -o app.js.map # Ferramenta: sourcemapper go install github.com/denandz/sourcemapper@latest sourcemapper -url https://alvo.com/static/app.js.map -output ./src_recuperado/ # Ferramenta: source-map-explorer npm install -g source-map-explorer source-map-explorer app.js app.js.map # Ferramenta: webpack-sourcemap-extractor python3 wsxtr.py -u https://alvo.com/static/app.js.map -d ./output/ # Resultado: pasta com estrutura de arquivos do projeto original # Ex: /src/components/Login.jsx, /src/api/auth.js, /config/keys.js
```

O que voce encontra no codigo original

- Comentarios de desenvolvedor com informacoes internas
- Chaves de API e tokens hardcoded em arquivos de config
- Endpoints de API internos e rotas nao documentadas
- Logica de validacao (permite entender como bypassar)
- Credenciais de ambiente (se .env foi acidentalmente incluido)
- Nomes de variaveis e funcoes significativos (facilitam o pentest)
- TODO/FIXME comentarios revelando vulnerabilidades conhecidas

09 // RECONSTRUINDO DOCUMENTACAO DE APIs PRIVADAS

Muitos sites tem APIs sem documentacao publica. Atraves de interceptacao e analise de JS, e possivel reconstruir completamente a interface da API — endpoints, parametros, autenticacao e comportamento esperado.

Workflow Completo

```
PASSO 1 — Coleta automatica com Burp: a. Ative o Proxy + Spider b. Navegue TODAS as funcionalidades do site (autenticado) c. Export: Target → Site Map → Save selected items PASSO 2 — Extrair endpoints do JS: grep -rEo '"(/api/|/v[0-9]+/)[^"]+"' *.js | sort -u > endpoints.txt grep -rEo "'/[a-z0-9_-]+/[a-z0-9_-]+'" *.js | sort -u >> endpoints.txt PASSO 3 — Organizar por metodo: grep -B2 -A5 'fetch\|axios' *.js | grep -E '(GET|POST|PUT|DELETE|PATCH)' PASSO 4 — Documentar no Postman/Insomnia: Criar collection com todos os endpoints encontrados Testar cada um com dados validos e invalidos PASSO 5 — Gerar OpenAPI spec: Burp Extension: OpenAPI Generator mitmproxy2swagger: converte dump do mitmproxy em spec
```

mitmproxy2swagger — Gerar Spec Automaticamente

```
# 1. Capturar trafego com mitmproxy mitmproxy -w dump.mitm # 2. Navegar o site completamente enquanto o proxy roda # 3. Converter para OpenAPI pip install mitmproxy2swagger mitmproxy2swagger -i dump.mitm -o api_spec.yml \ -p https://alvo.com/api/ --examples # 4. Visualizar a spec gerada npx @redocly/cli preview-docs api_spec.yml # Resultado: documentacao completa da API privada!
```

Ferramentas Complementares

Ferramenta	Funcao
Burp Suite	Interceptacao, modificacao, fuzzing de requisicoes HTTP/S
mitmproxy	Proxy CLI/Python para captura e modificacao programatica
Postman / Insomnia	Organizar e testar endpoints da API reconstruida
mitmproxy2swagger	Converter trafego capturado em especificacao OpenAPI
SecretFinder	Extrair chaves e endpoints de arquivos JavaScript
LinkFinder	Descobrir endpoints de API embutidos em JS
sourcemapper	Recuperar codigo-fonte original de source maps
jwt_tool	Analisar, modificar e crackar tokens JWT
GraphQL Voyager	Visualizar schema GraphQL graficamente
wafw00f	Detectar e fingerprint de WAF

10 // CHECKLIST PRATICO

[RECONHECIMENTO INICIAL]

- [ ] Identificar stack tecnologico (whatweb, Wappalyzer, headers)
- [ ] Verificar se ha source maps expostos (.js.map)
- [ ] Checar robots.txt e sitemap.xml
- [ ] Buscar repositorio Git exposto (/.git/config)
- [ ] Google Dorks para JS files e endpoints indexados

[ANALISE DE JAVASCRIPT]

- [ ] Baixar todos os arquivos JS do site
- [ ] Unminificar com js-beautify ou DevTools Pretty Print
- [ ] Buscar por segredos: api_key, secret, password, token
- [ ] Extrair endpoints de API embutidos no codigo
- [ ] Verificar bibliotecas vulneraveis (retire.js, npm audit)
- [ ] Analisar ofuscacao e tentar desofuscar

[INTERCEPTACAO DE REDE]

- [ ] Configurar Burp Suite / mitmproxy
- [ ] Navegar todo o site autenticado e nao autenticado
- [ ] Mapear todos os endpoints de API encontrados
- [ ] Analisar headers de autenticacao (Bearer, Cookie, Basic)
- [ ] Verificar WebSocket se presente
- [ ] Tentar introspeccao GraphQL se aplicavel

[TOKENS E AUTENTICACAO]

- [ ] Decodificar JWT e analisar payload
- [ ] Verificar algoritmo JWT (HS256, RS256, none)
- [ ] Analisar flags de cookies (HttpOnly, Secure, SameSite)
- [ ] Mapear fluxo OAuth se presente
- [ ] Verificar se tokens expiram e sao invalidados

[SOURCE MAPS E CODIGO]

- [ ] Verificar comentario //# sourceMappingURL= no JS
- [ ] Tentar acessar .map diretamente na URL
- [ ] Usar sourcemapper para extrair codigo original
- [ ] Analisar estrutura de arquivos recuperada
- [ ] Buscar credenciais e segredos no codigo original

[SPAs E FRAMEWORKS]

- [ ] Instalar React/Angular/Vue DevTools
- [ ] Inspecionar arvore de componentes e props/state
- [ ] Extrair rotas da SPA do codigo compilado
- [ ] Verificar Redux/Vuex store para dados sensiveis
- [ ] Analisar Service Workers (cache, offline mode)

[DOCUMENTACAO E RELATORIO]

- [ ] Organizar endpoints descobertos no Postman

- [ ] Gerar spec OpenAPI com mitmproxy2swagger
- [ ] Documentar segredos e informacoes sensiveis encontrados
- [ ] Classificar severidade de cada achado
- [ ] Preparar PoC para cada vulnerabilidade